



Relatório Semanal de Atividades – Semana 1

Projeto: Desenvolvimento de um Sistema Alternativo ao PASCO (Flexduino)

ROBERT NICOLAS DUARTE BASTOS
Nº USP: 15641639

19 de Outubro de 2025

Orientadores:

Prof. Alexandre Alarcon do Passo Suaide
Profa. Kaline R. S. Coutinho

1 Introdução e Justificativa

Este relatório inicia a documentação semanal do progresso no projeto Flexduino. O objetivo desta primeira semana foi estabelecer um fluxo de trabalho validado, começando pela verificação e depuração do firmware principal em um ambiente controlado antes de avançar para o hardware físico.

A decisão de usar um simulador (Wokwi) nesta fase é estratégica: permite isolar problemas de software de possíveis falhas de hardware (ex: mau contato, componente queimado). Ao garantir que o código se comporta como esperado em um ambiente ideal, ganhamos confiança para as fases seguintes e aceleramos o ciclo de desenvolvimento.

2 Metodologia e Execução

O processo foi dividido em duas fases principais: a depuração do firmware e a criação de um banco de testes virtual.

2.1 Fase 1: Depuração do Firmware

O ponto de partida foi o código-fonte flex_duino_1_2025.ino.

2.1.1 Etapa 1.1: Primeiro Erro de Compilação

A compilação inicial falhou com o erro 'POSITIVE' was not declared in this scope.

Análise Didática: Em programação, "scope" (escopo) refere-se à área onde uma variável ou constante é reconhecida. O erro indicou que a palavra POSITIVE, usada na configuração do display LCD, não existia naquele contexto. Isso acontece porque POSITIVE não é uma palavra-chave padrão da linguagem C++, mas sim uma constante específica definida dentro dos arquivos da biblioteca LiquidCrystal_I2C. A versão da biblioteca utilizada pelo simulador Wokwi pode ser diferente da versão usada no desenvolvimento original do código, ou pode não carregar essa definição específica da mesma forma. Essencialmente, o programa não sabia o que "POSITIVE" significava.

Solução: A solução foi "ensinar" ao programa o significado da constante antes que ele precisasse usá-la. Isso foi feito adicionando a linha #define POSITIVE 1 ao código, que funciona como uma instrução direta ao compilador: "onde quer que você veja a palavra POSITIVE, substitua-a pelo número 1".

2.1.2 Etapa 1.2: Segundo Erro de Compilação

Corrigido o primeiro erro, um segundo surgiu: unterminated comment.

Análise: Este erro significa que o compilador encontrou um início de comentário (/*) mas não encontrou seu fechamento (*/*).

Solução: Uma inspeção no código revelou um bloco de comentário que não havia sido fechado. A correção foi simplesmente adicionar o */ faltante.

2.1.3 Etapa 1.3: Sucesso na Comunicação

Após as correções, o firmware compilou com sucesso. O Monitor Serial passou a exibir a mensagem de status v1-2025, validando a comunicação unidirecional (Arduino -> Computador).

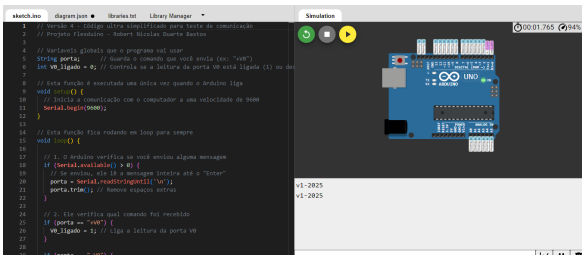


Figura 1: Teste de comunicação unidirecional bem-sucedido.

Em seguida, o comando +V0 foi enviado pelo terminal. O sistema prontamente começou a enviar a leitura da porta A0, confirmando a comunicação bidirecional.

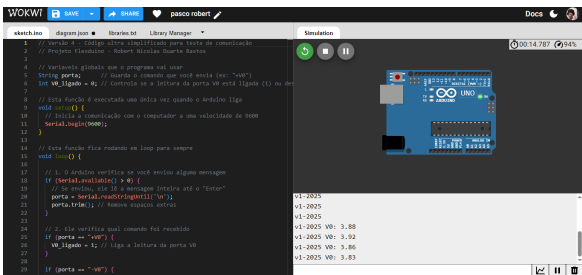


Figura 2: Validação da comunicação bidirecional.

2.2 Fase 2: Banco de Testes Virtual

Os dados da porta A0 estavam "flutuando". Para criar um cenário de teste realista, um sensor virtual (potenciômetro) foi conectado aos pinos 5V, GND e A0 do Arduino. Ao variar o potenciômetro, os valores no Monitor Serial responderam em tempo real de 0.00V a 5.00V de forma controlada.

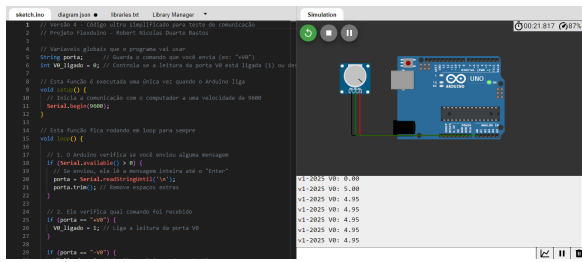


Figura 3: Banco de testes com sensor virtual operacional.

2.3 Código-Fonte Utilizado

Para isolar a funcionalidade de comunicação, foi utilizada uma versão simplificada do firmware. Este código foca exclusivamente em ler comandos e reportar o estado da porta A0.

```

1 // Versao 4 - Codigo ultra simplificado
2 // para teste de comunicacao
3 String porta;
4 int V0_ligado = 0;
5
6 void setup() {
7     Serial.begin(9600);
8 }
9
10 void loop() {
11     if (Serial.available() > 0) {
12         porta = Serial.readStringUntil('\n');
13         porta.trim();
14     }
15
16     if (porta == "+V0") {
17         V0_ligado = 1;
18     }
19
20     if (porta == "-V0") {
21         V0_ligado = 0;
22     }
23
24     Serial.print("v1-2025");
25
26     if (V0_ligado == 1) {
27         int sensorValue0 = analogRead(A0);
28         float voltage0 = sensorValue0 * (5.0 / 1023.0);
29         Serial.print(" V0: ");
30         Serial.print(voltage0);
31     }
32
33     Serial.println();
34     porta = "";
35     delay(1000);
36 }

```

Listing 1: Versão 4 - Firmware de teste de comunicação.

3 Resultados e Conclusões

O trabalho da semana foi um sucesso. Foi estabelecido um ambiente de teste funcional que permitiu validar o firmware e simular a aquisição de dados. A criação deste banco de testes virtual é um marco importante, pois permite que o software em Python seja desenvolvido e testado sem a necessidade de acesso imediato ao hardware físico, otimizando o cronograma.

4 Próximos Passos e Justificativa

O objetivo para a próxima semana é iniciar a automação do processo no lado do computador.

4.1 Justificativa da Metodologia

A estratégia de desenvolver a interface em Python *antes* de realizar testes com os sensores físicos é adotada por eficiência. O teste desta semana já validou que o firmware responde aos comandos. Agora, ao construir a interface de usuário, criaremos uma ferramenta visual (com gráficos e botões) para realizar os futuros testes com os sensores físicos. Testar um sensor vendo seus dados plotados em um gráfico é muito mais eficaz do que apenas ver números em um terminal.

4.2 Meta para a Próxima Semana

O foco será criar um programa simples em Python que consiga:

1. Conectar-se à porta serial do Arduino (virtual).
2. Enviar o comando +V0 automaticamente.
3. Receber as respostas e imprimi-las no terminal.